SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

For
Project: Hues 'n Dash
(Open-Source 2D Puzzle-Platformer Using Godot Engine)

Version: 1.0
Date: February 12, 2026

Submitted By
Aatish Ashok A
23TD0001
Computer Science Engineering

Academic Year
2025 – 2026

Table of Contents

INTRODUCTION **1**

1.1 Purpose 1

1.2 Scope 1

1.3 Definitions, Acronyms, and Abbreviations 2

1.4 References 2

OVERALL DESCRIPTION **3**

2.1 Product Perspective 3

2.2 Product Functions 3

2.3 User Characteristics 3

2.4 Operating Environment 3

2.5 Design and Implementation Constraints 4

2.6 Assumptions and Dependencies 4

SPECIFIC REQUIREMENTS **4**

3.1 Functional Requirements 4

3.2 Non-Functional Requirements 5

EXTERNAL INTERFACE REQUIREMENTS **6**

4.1 User Interface 6

4.2 Audio Interface 6

SYSTEM RESOURCES **6**

5.1 Software Resources 6

5.2 Hardware Resources 6

DEVELOPMENT METHODOLOGY **6**

FUTURE ENHANCEMENTS **7**

CONCLUSION **7**

1 INTRODUCTION

1.1 PURPOSE

The purpose of this Software Requirements Specification (SRS)document is to define the functional and non-functional requirements of an open-source single-player 2D puzzle-platformer game. This document serves as a formal reference for development, evaluation, testing, and academic assessment.

1.2 SCOPE

The project involves the development of a F2P 2D puzzle-platformer game under the Multimedia Systems (Game Development) domain using the Godot Engine. The game is designed for offline single-player gameplay and follows the Agile software development model.

the core gameplay mechanics include:

- Color-coded items and doors
- Single-item carrying constraint
- Enemy characters that pursue the player
- Time-limited levels

The game will be developed using open-source tools and deployed on the itch.io platform.

1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS

Term	Description
SRS	Software Requirements Specification
GDD	Game Desing Document
PC	Personal Computer
OS	Operating System
UI	User Interface
RTS	Real-Time Strategy
F2P	Free To Play

1.4 References

1. Godot Engine Documentation – https://docs.godotengine.org
2. Sommerville, I., *Software Engineering*, Pearson Education
3. Pressman, R. S., *Software Engineering: A Practitioner's Approach*
4. Pixabay – https://pixabay.com
5. itch.io – https://itch.io
6. Krita – https://krita.org
7. Ibis Paint – https://ibispaint.com

2 Overall Description

2.1 Product Perspective

The game is a standalone software application developed using the Godot Engine. It does not depend on any external systems or online services. The game will be deployed and distributed through itch.io as an open-source project.

2.2 Product Functions

The major functions of the system include:

- Player movement and platform navigation
- Item collection and management (single item constraint)
- Color-coded door access mechanism
- Enemy character that follows the player
- Time-limited gameplay per level
- Level completion detection
- Audio and visual feedback
- Game restart and level progression

2.3 User Characteristics

The target users are casual players with basic familiarity with keyboard controls.

No prior technical knowledge is required to play the game.

2.4 Operating Environment

- Platform: Windows
- Engine: Godot
- Execution Mode: Offline
- Input Devices: Keyboard

2.5 Design and Implementation Constraints

- The game follows the Agile development model
- Single-player only
- Offline gameplay
- Open-source codebase
- Limited scope suitable for a mini project

2.6 Assumptions and Dependencies

- The user has a system capable of running Godot-based games
- Required assets are royalty-free or self-created
- The game does not require internet connectivity after installation

3 Specific Requirements

3.1 Functional Requirements

FR-1: Player Movement

The system shall allow the player to move left, right, jump, and interact with objects within the game environment.

FR-2: Level Structure

The system shall support multiple levels, each with a defined start point and finish point.

FR-3: Item Collection

The system shall allow the player to collect one interactive item at a time.

FR-4: Single Item Constraint

The system shall restrict the player to carrying only one item at any given time.

FR-5: Color-Coded Access Control

The system shall allow the player to pass through a door only if the carried item matches the door's color.

FR-6: Enemy Behaviour

The system shall include enemy characters that actively follow or chase the player within the level.

FR-7: Time Constraint

The system shall impose a time limit for each level, requiring the player to complete the level before time expires.

FR-8: Level Completion

The system shall mark the level as completed when the player reaches the finish point under valid conditions.

FR-9: Failure Conditions

The system shall restart or end the level if the player is caught by an enemy or the time limit expires.

3.2 Non-Functional Requirements

NFR-1: Performance

The game shall run smoothly without noticeable lag on standard desktop systems.

NFR-2: Usability

The game shall have simple and intuitive controls suitable for casual players.

NFR-3: Reliability

The game shall function consistently without crashes during gameplay.

NFR-4: Maintainability

The source code shall be modular and well-structured to allow future updates and improvements.

NFR-5: Portability

The game shall be deployable on multiple desktop platforms supported by Godot.

4 External Interface Requirements

4.1 User Interface

- Visual UI elements such as buttons, timers, and indicators
- UI assets sourced from itch.io
- Clear visual feedback for item collection and door access

4.2 Audio Interface

- Background music and sound effects
- Royalty-free audio assets sourced from Pixabay

5 System Resources

5.1 Software Resources

- **Godot Engine** – Game development
- **Git & GitHub** – Version control and repository hosting
- **Krita** – Artwork creation
- **Ibis Paint** – Artwork creation
- **Pixabay** – Royalty-free audio assets
- **itch.io** – Deployment and distribution platform

5.2 Hardware Resources

- Desktop or laptop computer
- Keyboard input device

6 Development Methodology

The project follows the Agile development model, where development is carried out in iterative cycles. Each iteration includes requirement analysis, design, implementation, testing, and review. Feedback from earlier iterations is used to improve subsequent versions of the game.

7 FUTURE ENHANCEMENTS

The following features are planned for future versions of the game:

- Introduction of additional enemy types with unique movement and attack patterns
- Implementation of advanced puzzle mechanics (moving platforms, timed switches, pressure plates, teleporters, etc.)
- Addition of more levels with progressively increasing difficulty
- Enhanced visual effects and character animations
- Cross-platform builds (Windows, Linux, Web, possibly Android)

8 CONCLUSION

This SRS document defines the complete functional and non-functional requirements for the 2D puzzle-platformer game. It serves as a foundation for development, testing, and evaluation of the project while maintaining clarity and alignment with academic expectations.